

Project Title

AI-Driven Bug Report Classification and Severity Prediction System

AUTHORS

Bhavana Battala

Priyanka Mirgule

Sai Deepthi K

Shriya Deshpande

ABSTRACT

Software maintenance teams rely heavily on bug reports to identify defects, prioritize issues, and plan corrective actions. Large-scale software systems generate thousands of unstructured bug reports, making manual triaging slow, inconsistent, and resource-intensive.

This project proposes an AI-driven system for automated bug report classification and severity prediction using Natural Language Processing (NLP) and machine learning techniques. The system analyzes textual bug reports and classifies them into categories (e.g., UI, performance, security) while simultaneously predicting their severity levels (e.g., low, medium, high, critical). Predicting severity is particularly challenging because it depends on contextual cues beyond simple keyword presence.

The model is trained on a dataset of 50,000 bug reports and implements multiple approaches, including traditional machine learning (TF-IDF with Logistic Regression and SVM), deep learning (Text-CNN), and transformer-based models (BERT and DistilBERT). Transformer models leverage contextual embeddings and self-attention mechanisms to capture semantic meaning and long-range dependencies, which improves classification accuracy compared to simpler models.

Comparative evaluation using accuracy, precision, recall, and F1-score demonstrates that transformer-based models outperform traditional and CNN-based methods. The proposed system reduces manual effort, improves bug triaging efficiency, and provides a more consistent prioritization of software defects, making it valuable for modern software maintenance workflows.

1. INTRODUCTION

Domain and Significance:

Software engineering and maintenance heavily rely on bug reports to identify, analyze, and resolve defects in software systems. Large-scale applications often generate thousands of unstructured textual bug reports, making it difficult for manual triaging to keep pace. Efficient bug triaging is critical for maintaining software quality, enhancing user experience, and reducing system downtime. As software systems grow in size and complexity, the volume of bug reports increases, highlighting the need for intelligent automation to support software maintenance teams effectively.

Problem Statement:

Bug reports are typically unstructured and require manual analysis to determine both the type of defect and its severity. This process is time-consuming, inconsistent, and difficult to scale as the number of reports increases. Consequently, critical bugs may be delayed, development teams face increased workload, and overall efficiency in managing and prioritizing issues is reduced. An automated system capable of classifying bug types and predicting their severity can significantly improve software maintenance workflows.

Existing Solutions:

Traditional approaches for bug classification rely on machine learning techniques, such as TF-IDF combined with classifiers like Logistic Regression and Support Vector Machines. More recent methods apply deep learning models, including Convolutional Neural Networks (CNNs), to capture textual patterns and sequential information. Additionally, transformer-based models such as BERT have been applied for improved contextual understanding. However, most existing solutions focus on either bug classification or severity prediction alone and often lack a comprehensive comparative analysis across multiple approaches. There is also limited research on integrating both bug classification and severity prediction into a single unified system.

Proposed Solution:

- Develop an AI-based system to automatically classify bug reports and predict their severity (two-stage prediction system).
- Combine bug report titles and descriptions as input with preprocessing for better text quality.
- Implement multiple approaches: TF-IDF with Logistic Regression and SVM, Text-CNN, BERT, and DistilBERT.
- Evaluate and compare models using standard metrics such as accuracy, precision, recall, and F1-score.

- Improve the efficiency and consistency of the bug triaging process in software maintenance workflows.

Contributions:

- Development of an automated system for both bug classification and severity prediction.
- Implementation of multiple models, including traditional machine learning, deep learning, and transformer-based approaches (BERT and DistilBERT).
- Comparative analysis of different models to evaluate their effectiveness on textual bug report data.
- Utilization of a large-scale dataset (50K bug reports) to validate the proposed system.
- Enhancement of bug triaging efficiency and consistency for software maintenance teams.

2. RELATED WORK

- Several studies have explored automatic bug classification using traditional machine learning techniques.
- Early approaches primarily relied on text mining and feature engineering using TF-IDF representations.
- These approaches applied classifiers such as Logistic Regression, Naïve Bayes, and Support Vector Machines for categorizing bug reports.
- The results demonstrated that classical models can achieve strong performance when bug descriptions are structured and consistent.
- With the advancement of deep learning, researchers began using neural network models for bug report classification.
- Convolutional Neural Networks (CNNs) were applied to capture local contextual patterns in bug descriptions.
- Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) models were also explored for sequential text understanding.
- These models improved semantic feature extraction compared to traditional methods.
- Recently, transformer-based models have gained attention for software engineering text classification tasks.
- Models such as BERT and DistilBERT have been used for bug categorization and severity prediction.
- These models leverage contextual embeddings and pre-trained language representations.
- Transformer-based approaches generally outperform traditional models when large datasets are available.
- Some studies have also investigated multi-task learning for predicting both bug category and severity simultaneously.
- These approaches demonstrated that shared representations can improve classification accuracy.

- However, multi-task models often require large labeled datasets and higher computational resources.
- Despite these advances, few studies provide a comprehensive comparison of traditional machine learning, deep learning, and transformer-based models on the same dataset.
- Additionally, limited work evaluates both bug category classification and severity prediction together.
- This study addresses these gaps by comparing multiple models under a unified experimental setup.

3. DATA

3.1 Data Collection and Preprocessing

A. Data Sources

- The project uses a dataset of 50,000 bug reports stored in a CSV file (bug_dataset_50k.csv).
- Each record contains multiple columns including bug_id, title, description, bug_category, severity, root_cause, suggested_fix, among others.
- For this project, only the essential columns were used: bug_id, title, description, bug_category, severity, root_cause, and suggested_fix.

B. Preprocessing Steps

a. Data Cleaning:

- **Missing Values:** All rows containing missing values in key columns were removed.
- **Duplicates:** Duplicate entries were dropped to ensure data quality.
- **Text Cleaning:** The title and description fields were combined and cleaned by:
 - Converting text to lowercase.
 - Removing common irrelevant words such as “bug”, “error”, “issue”, “detected”, and “system”.
 - Removing special characters and extra whitespace.
- After cleaning, records with very short text were filtered out to ensure meaningful input for modeling.

b. Label Encoding:

- Categorical labels bug_category and severity were encoded into numerical values using LabelEncoder.
- Example: API Bug → 0, Authentication Bug → 1, ... and Critical → 0, High → 1, etc.

c. Train-Test Split:

- The dataset was split into training and testing sets using **stratified sampling** to maintain the distribution of bug categories.
- For **transformer-based models (BERT and DistilBERT)**, a subset of **1,500 training samples and 500 test samples** was used due to computational constraints.

d. Tokenization and Feature Extraction:

- **For traditional ML models (Logistic Regression, SVM):**
 - Text was converted into numerical features using TF-IDF vectorization with 5,000 maximum features.
- **For CNN models:**
 - Text was tokenized into sequences and padded to a fixed length of 100 tokens.
 - A vocabulary size of 10,000 was used.
- **Transformer-Based Models (BERT and DistilBERT):**
 - **BERT:** Texts were tokenized using “bert-base-uncased” tokenizer with a maximum sequence length of 32 tokens.
 - **DistilBERT:** Texts were tokenized using “distilbert-base-uncased” tokenizer with a maximum sequence length of 32 tokens.
 - Data was converted into Hugging Face Dataset format, and **DataCollatorWithPadding** was used for batching.

Preprocessing was performed to convert noisy and inconsistent textual bug reports into a cleaner and more standardized form suitable for machine learning. Since bug reports are typically written by different users with different styles, preprocessing helps reduce unnecessary variation while preserving the semantic content relevant to classification.

4. METHODS

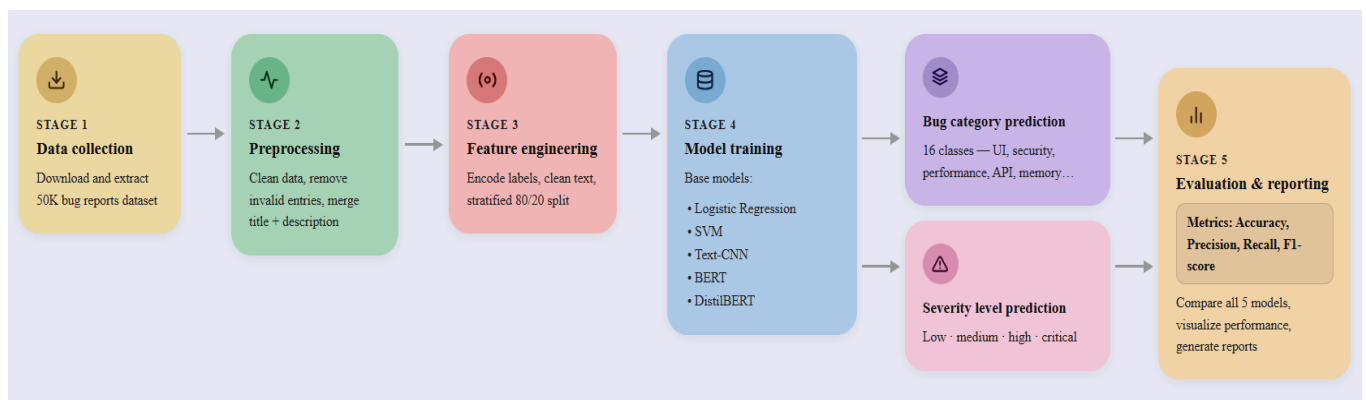


Figure 1: Overview of the Bug Report Classification and Severity Prediction Pipeline.

The figure illustrates the complete workflow of the proposed system, including data collection, preprocessing, feature engineering, model training using Logistic Regression, SVM, CNN, BERT and DistilBERT, and evaluation using accuracy, precision, recall, and F1-score.

4.1 Model Selection and Training

Overview:

The project implements and compares multiple approaches for automatic bug classification: traditional machine learning, deep learning (CNN), and transformer-based models (BERT and DistilBERT). The aim is to evaluate how different models capture textual patterns from bug reports and classify them into predefined categories.

1. Traditional Machine Learning Approaches:

- **Feature Extraction:**
 - Combined title and description fields from the bug reports.
 - Cleaned text by lowercasing, removing special characters, and removing common words like “bug”, “error”, and “detected”.
 - Converted text into numerical vectors using **TF-IDF (Term Frequency-Inverse Document Frequency)** with a maximum of 5,000 features.
- **Models Used:**
 - **Logistic Regression** - optimized with max_iter=1000.
 - **Support Vector Machine (SVM)** - linear kernel for text classification.
- **Objective:** Establish baseline performance and evaluate the effectiveness of traditional ML on bug report text

2. Deep Learning Approach (CNN):

- **Text Preparation:**
 - Tokenized text sequences using Tokenizer and padded sequences to a **maximum length of 100 tokens**.
 - Vocabulary size limited to 10,000 to cover frequently used words.
- **Model Architecture:**
 - Embedding layer with 100 dimensions.
 - **1D Convolutional layer (Conv1D)** with 128 filters and kernel size 5.
 - Global Max Pooling layer to reduce sequence dimensionality.
 - Fully connected Dense layers with softmax output for multi-class classification.
- **Training Details:**
 - Optimizer: Adam
 - Loss: Sparse Categorical Crossentropy
 - Epochs: 5, Batch size: 32

3. Transformer-Based Approaches (BERT and DistilBERT)

- **Tokenizer & Preprocessing:**

- Used bert-base-uncased and distilbert-base-uncased tokenizers.
 - Maximum sequence length: 32 tokens.
- **Model:** Fine-tuned AutoModelForSequenceClassification for bug categories and severity labels.
- **Training Setup:**
 - Learning rate: 5e-5
 - Batch size: 32
 - Weight decay: 0.01
 - Epochs: 5
 - Hugging Face Trainer API used for training, evaluation, and metrics computation.
- **Objective:** Leverage contextual embeddings to understand complex bug descriptions better than traditional methods.

4. Evaluation Metrics:

- Performance evaluated using **accuracy, precision, recall, and F1-score** (weighted to account for class imbalance) for each bug category.
- Comparison shows the effect of **feature representation** (TF-IDF vs embeddings) and **model type** (traditional, deep learning, transformer) on classification performance.

Frameworks & Libraries:

- **Python Libraries:** pandas, numpy, scikit-learn, tensorflow/keras, transformers, datasets
- **Frameworks:** Hugging Face Transformers for BERT and DistilBERT, Keras for CNN, scikit-learn for traditional ML.

5. EXPERIMENTS AND RESULTS

The project was designed as a comparative machine learning study in which multiple models were trained and evaluated under a common preprocessing and reporting framework. Two supervised prediction tasks were considered: bug category classification and severity prediction. Each model was assessed on its ability to learn meaningful patterns from the bug report text and generalize to unseen reports.

The experimental design included three major model families:

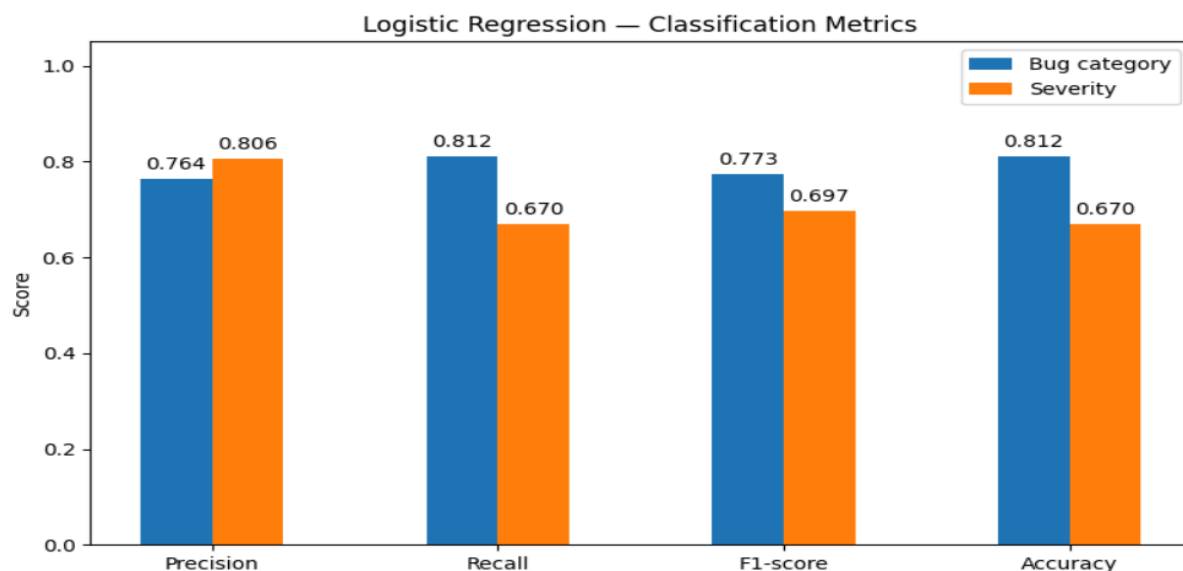
1. Traditional machine learning models using TF-IDF features
2. Deep learning models using learned word embeddings and convolutional layers
3. Transformer-based models using contextual embeddings from pre-trained language models

The AI-driven bug report classification system was evaluated using five modeling approaches: Logistic Regression, Support Vector Machine (SVM), Text-CNN, BERT, and DistilBERT. All

models were trained and tested on preprocessed bug report text created by combining titles and descriptions. The evaluation was conducted using accuracy, precision, recall, and F1-score, along with qualitative analysis using example predictions.

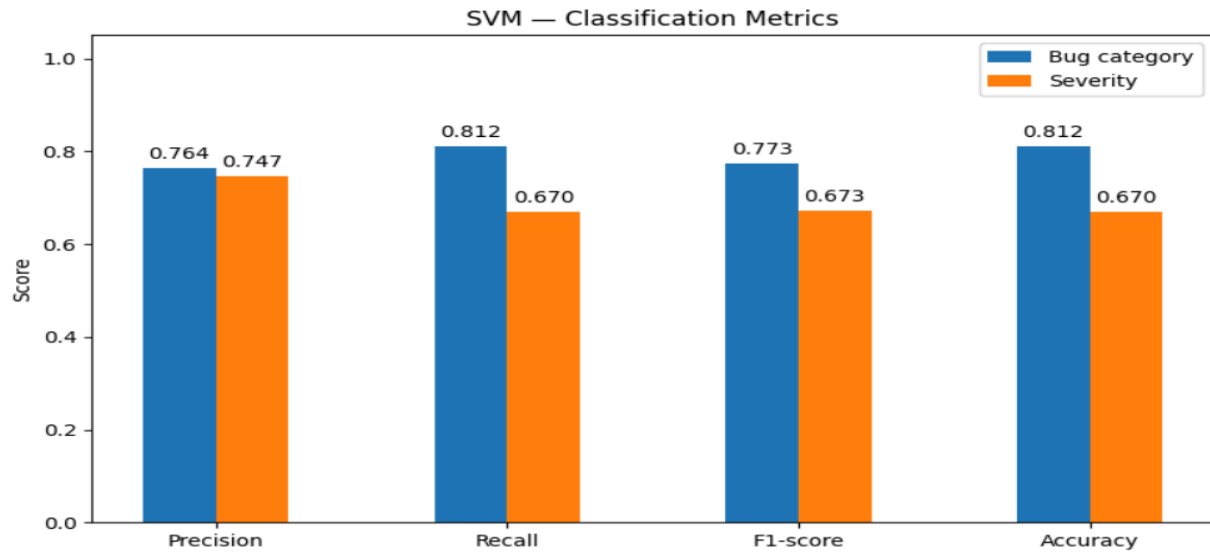
Logistic Regression:

- For **bug category** classification, Logistic Regression achieved an accuracy of **0.8116**.
- Weighted precision, recall, and F1-score were **0.76**, **0.81**, and **0.77**, respectively.
- The model performed well for clearly separable categories but struggled with context-dependent reports.
- For **severity prediction**, Logistic Regression achieved an accuracy of **0.6663**.
- Weighted precision, recall, and F1-score were approximately **0.83**, **0.67**, and **0.70**, respectively.
- The model showed moderate performance in predicting severity levels, with better prediction for high-frequency severity classes and lower performance for minority classes



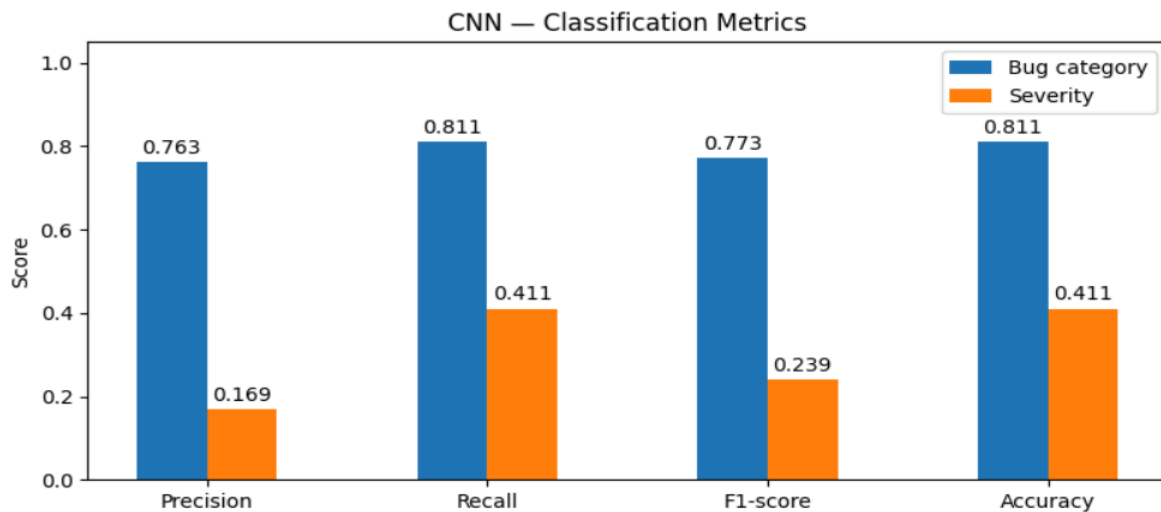
Support Vector Machine (SVM):

- For **bug category** classification, SVM achieved an accuracy of **0.8116**.
- Weighted precision, recall, and F1-score were **0.76**, **0.81**, and **0.77**, respectively.
- The model produced performance similar to Logistic Regression and handled high-dimensional TF-IDF features effectively.
- For **severity prediction**, SVM achieved an accuracy of **0.67**.
- Weighted precision, recall, and F1-score were approximately **0.75**, **0.67**, and **0.67**, respectively.
- The model showed stable performance across severity levels but struggled with distinguishing between medium and low severity categories.



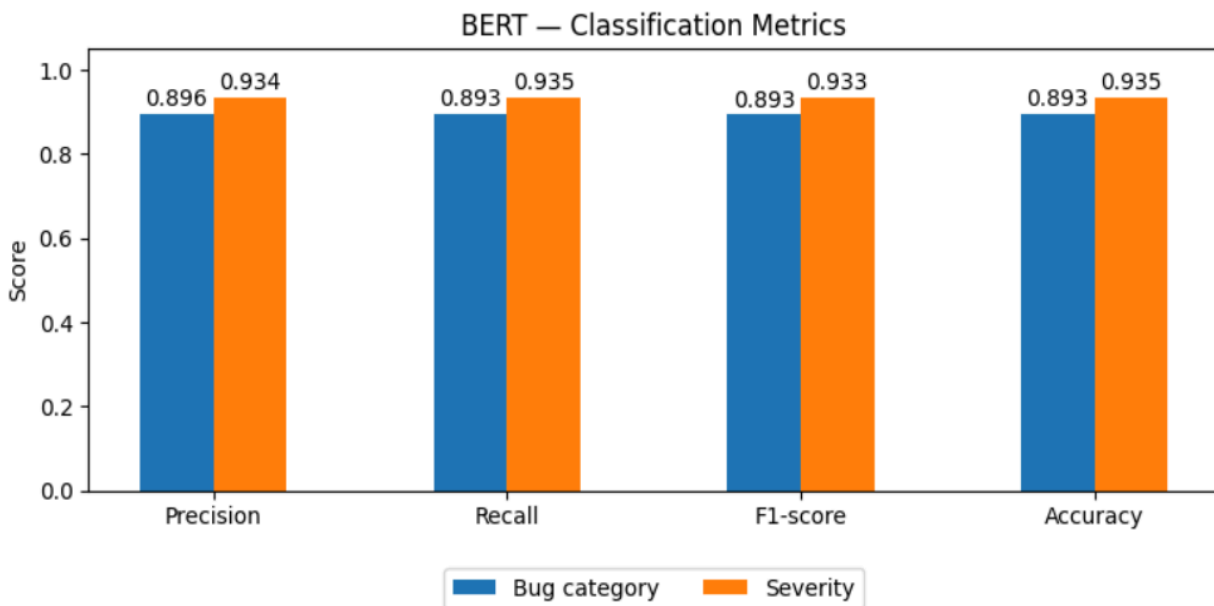
Text-CNN:

- For **bug category** classification, the Text-CNN model achieved an accuracy of **0.811**.
- Weighted precision, recall, and F1-score were **0.76**, **0.81**, and **0.77**, respectively.
- The model captured local sequential patterns in bug descriptions.
- For **severity prediction**, the Text-CNN model achieved an accuracy of **0.41**.
- Weighted precision, recall, and F1-score were approximately **0.17**, **0.41** and **0.24**
- However, CNN showed weak performance for severity prediction, suggesting that local phrase detection alone is insufficient for capturing contextual signals needed for severity estimation.



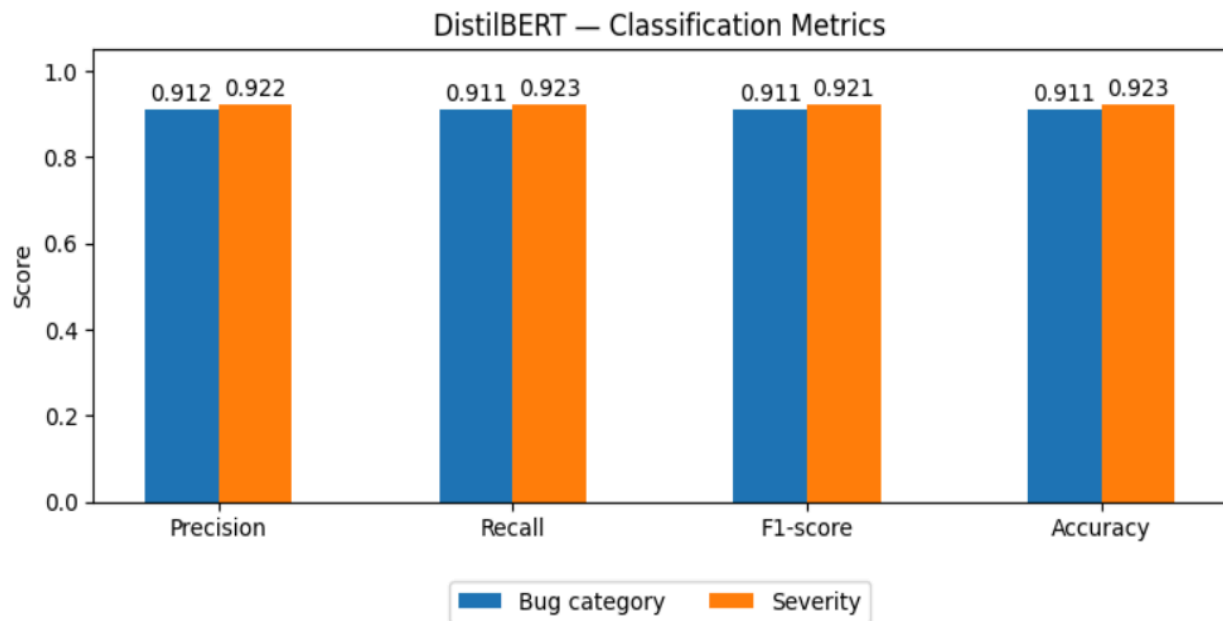
BERT:

- For **bug category** classification, BERT achieved an accuracy of **0.893**.
- Weighted precision, recall, and F1-score were **0.896**, **0.892**, and **0.893**, respectively.
- The model demonstrated strong performance in capturing complex textual patterns and outperformed traditional ML models, especially on imbalanced categories.
- For **severity prediction**, BERT achieved an accuracy of **0.935**.
- Weighted precision, recall, and F1-score were **0.922**, **0.868**, and **0.892**, respectively.
- The model handled severity distinctions well, though it showed slightly lower recall for medium-severity cases compared to high-severity cases.



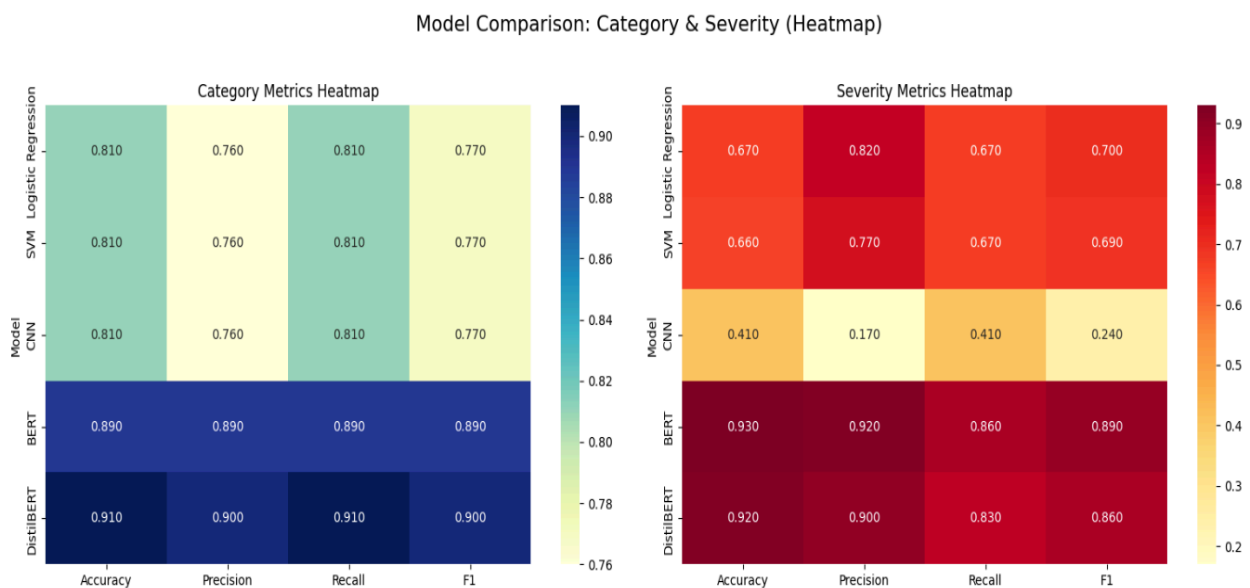
DistilBERT:

- For **bug category** classification, DistilBERT achieved an accuracy of **0.911**.
- Weighted precision, recall, and F1-score were **0.910**, **0.911**, and **0.910**, respectively.
- The model delivered strong performance while being more computationally efficient than BERT, making it suitable for faster inference on large datasets.
- For **severity prediction**, DistilBERT achieved an accuracy of **0.923**.
- Weighted precision, recall, and F1-score were **0.908**, **0.837**, and **0.868**, respectively.
- The model performed reliably across severity levels, although recall was slightly lower for medium-severity cases, similar to BERT.



Summary:

Transformer-based models (BERT and DistilBERT) outperformed traditional ML models and Text-CNN in both bug category and severity classification. DistilBERT achieved the highest category accuracy (0.911) while being more computationally efficient than BERT. For severity prediction, BERT achieved the highest accuracy (0.935), though DistilBERT delivered comparable performance. Traditional models and Text-CNN performed moderately, struggling with complex or low-frequency cases.



5.1 Confusion Matrix Analysis

Beyond accuracy, confusion matrix analysis provides deeper insight into model behavior.

- For **traditional models** (Logistic Regression and SVM), misclassifications typically occurred between similar categories, such as **API Bug vs. Backend Bug**, indicating that these models capture general patterns but struggle with subtle semantic differences.
- **Text-CNN** demonstrated better separation of categories by detecting local patterns in the text, allowing for improved classification of recurring bug phrases like “memory leak” or “timeout.”
- **BERT** showed strong diagonal dominance in the confusion matrix, reflecting superior classification for complex and ambiguous bug reports. It effectively captures full-sentence context, which helps distinguish subtle category differences.
- **DistilBERT**, while slightly less precise than BERT in some metrics, DistilBERT captures semantic relationships more efficiently than traditional ML models and Text-CNN. Its ability to leverage sentence-level context enables better understanding of complex, context-rich bug reports, making it suitable for faster inference with comparable performance.

5.2 Comparative Insights

The comparative analysis reveals that no single model is universally best under all conditions. Instead, each model family offers distinct strengths and weaknesses.

- **Logistic Regression** and **SVM** performed strongly because TF-IDF features captured informative terms and patterns that are highly relevant for bug categorization. These models were computationally efficient, easier to train, and more interpretable than neural alternatives.
- The **Text-CNN** model was able to capture local phrase-level patterns in the data, making it more expressive than traditional bag-of-words methods. However, its performance on severity prediction was weaker, suggesting that local phrase detection alone may be insufficient for fully capturing the nuanced contextual meaning required for severity estimation.
- **BERT** and **DistilBERT** leveraged advanced contextual representations of language, achieving the highest classification accuracy for both bug category and severity. DistilBERT provided slightly higher category accuracy (0.911) and faster computation, while BERT achieved the highest severity accuracy (0.935). Some metrics, such as medium-severity recall, were slightly lower, highlighting the need to align model choice with available data, computational resources, and deployment goals.

6. CONCLUSIONS

This project developed an AI-based system to classify bug reports and predict severity using multiple machine learning models. The results showed that transformer-based models, particularly BERT and DistilBERT, achieved the best overall performance, while Logistic Regression and SVM provided strong baseline results with lower computational cost. Overall, the study demonstrates that traditional machine learning methods are efficient and effective for baseline bug triaging, whereas transformer-based approaches offer superior accuracy and represent a strong option for further improvement with additional data and hyperparameter tuning.

a. Accomplishments:

- Developed an AI-driven system for automated **bug report classification and severity prediction** using five different modeling approaches: Logistic Regression, SVM, CNN, BERT and DistilBERT.
- Successfully preprocessed a large-scale dataset of 50,000 bug reports, including cleaning, tokenization, and label encoding.
- Conducted comprehensive experiments and evaluations, showing that transformer-based models (BERT and DistilBERT) achieved the highest accuracy for both bug category and severity, demonstrating the advantage of contextual embeddings over traditional and CNN-based approaches.
- Demonstrated that combining bug report titles and descriptions improves model understanding and classification performance.
- Validated the system's effectiveness in classifying both frequent and rare bug categories, enhancing **bug triaging efficiency** for software maintenance teams.

b. Limitations:

1. Limited BERT and DistilBERT Training:

Transformer models were fine-tuned on a small subset of the dataset due to computational constraints, which may have limited their performance.

2. Text-Only Features:

The system relies solely on bug report text and does not incorporate metadata such as developer history, environment, or error codes, which could improve predictions.

3. Rare or Ambiguous Bug Reports:

Some bug reports that are rare, ambiguous, or poorly described remain difficult to classify accurately, even for deep learning and transformer models.

4. Dataset Imbalance:

Certain bug categories are underrepresented, which can bias models toward more frequent categories and reduce accuracy for rare types. Class imbalance in severity labels contributed to lower performance for minority classes.

5. Short Sequence Length for Transformers:

BERT and DistilBERT were limited to 32-token sequences, potentially truncating important context in longer bug descriptions.

6. Generalizability:

The models were trained on a specific 50K bug report dataset; performance on bug reports from other domains or projects may differ, limiting generalizability.

c. Future Work:

- Extend BERT training to the entire dataset and explore larger transformer variants for improved performance.
- Incorporate additional features such as environment, error codes, and developer roles to enhance model predictions.
- Implement multi-task learning for a single model to predict both bug category and severity.
- Explore ensemble models, such as combining Logistic Regression and BERT, to improve classification accuracy.
- Implement real-time deployment using APIs for seamless integration into software development workflows.
- Integrate the system with tools like Jira and GitHub Issues for automated bug tracking and triaging.
- Investigate explainable AI techniques to help developers understand why a particular bug report was classified in a certain way.

7. DATA AVAILABILITY

All data can be found in <https://github.com/deepthi59/Bug-Report-Classification>

8. ACKNOWLEDGEMENTS

We would like to sincerely thank our professor, Dr. Tianle Ma, for their guidance, encouragement, and valuable feedback throughout this project. Their support helped us navigate challenges and improve the quality of our work.

We are grateful for the opportunity to collaborate as a project team, which enhanced our learning experience and allowed us to combine our skills effectively in problem-solving, data analysis, and model development.

We also appreciate the availability of the Bug report dataset on GitHub, which provided a rich foundation for our experiments, and the tools and resources such as Python, scikit-learn, TensorFlow/Keras, Hugging Face Transformers, and Google Colab, which made implementation and evaluation feasible.

This project provided us with practical experience in machine learning, deep learning, and transformer-based NLP models, and we are thankful for the structured guidance and resources that supported our academic growth.

9. REFERENCES

1. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv:1810.04805.
2. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv:1910.01108.
3. Le, Q., & Mikolov, T. (2014). *Distributed Representations of Sentences and Documents*. Proceedings of ICML, 1188–1196.
4. Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification*. arXiv:1408.5882.
5. Yasir Abdullah Mirza, “50K Bug Dataset,” *Kaggle*, 2026. [Online]. Available: <https://www.kaggle.com/datasets/mirzayasirabdullah07/50k-bug-dataset?resource=download>
6. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
7. Chollet, F. (2015). *Keras: Deep Learning Library for Theano and TensorFlow*. GitHub repository.
8. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP: Systems Demonstrations, 38–45.
9. Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.